

Formal Verification Report: Avant RequestManagerV2

- **Date:** June 2026
 - **Audit Repo:** <https://github.com/Cyfrin/audit-2026-06-avant-requestmanagerv2>
 - **Client Repo:** <https://github.com/Avant-Protocol/Avant-Contracts-Max>
 - **Audit Commit:** 0738f33ebc960522187546446e62e7037413abbc
 - **Mitigation Commit:** ec30a6ccba0def3e745b8c5ffb474cef39c45c90
 - **Author:** Julius Raynaldi (Cyfrin Audit External)
 - **Certora Prover version:** 8.16.1
-

Contents

Project Summary.....	1
Project Scope	1
Protocol Overview	2
Finding Summary	2
Detailed Findings	3
Low Severity	3
L-01 RequestsManagerV2::completeMint and RequestsManagerV2::completeBurn forwards de-allowlisted deposit tokens to treasury	3
Formal Verification	5
Formal Verification Methodology	5
Types of Properties	5
Verification Notations	5
General Assumptions and Simplifications	5
Verification Properties	6
RequestsManagerV2 — Access Control	6
RequestsManagerV2 — Idempotency Keys	7
RequestsManagerV2 — Escrow	8
RequestsManagerV2 — State Machine	9
RequestsManagerV2 — Structural	10
RequestsManagerV2 — Settlement Mathematics	11
SimpleToken — Core (Idempotency & Access Control)	12
SimpleToken — ERC-20	13
PriceStorage	14
Setup and Executions	16
Resources	19
Disclaimer	20

Project Summary

Project Scope

This document describes the specification and verification of Avant RequestManagerV2 using the Certora Prover. The work was undertaken from **June 14th 2026** to **June 19th 2026**.

The following contract list is included in our scope:

```
- src/RequestsManagerV2.sol
- src/interfaces/IRequestsManagerV2.sol
- src/PriceStorage.sol
- src/SimpleToken.sol
- src/interfaces/ISimpleToken.sol
- src/interfaces/IPriceStorage.sol
```

The Certora Prover demonstrated that the implementation of the Solidity contracts above is correct with respect to the formal rules written by **Julius Raynaldi** ([@JuliusRaynaldi](#)).

Protocol Overview

The protocol implements a token minting and burning system that bridges external assets into wrapped versions through a request-based mechanism. At its core, RequestsManager serves as the central orchestrator, accepting deposits of whitelisted tokens from approved providers who request minting of the wrapped token. These requests enter a pending state until a designated service role completes them with the final mint amount, which may be greater but not lesser than the requested amount due to exchange rates.

The system works bidirectionally - users can also burn their wrapped tokens to request redemption of the underlying assets, following the same request-complete pattern. The treasury address receives deposited tokens from completed mints and provides tokens for burns, maintaining clear fund flow separation from the operational contracts.

The architecture employs several supporting contracts to ensure operational security and flexibility:

- SimpleToken provides the ERC20 implementation with idempotent mint/burn functions that prevent double-processing through unique keys
- AddressesWhitelist maintains an optional KYC layer, allowing the protocol to restrict participation to verified addresses when regulatory compliance is required
- PriceStorage implements a price oracle system with configurable bounds checking, preventing extreme price movements between updates by enforcing upper and lower percentage limits on consecutive price changes

Finding Summary

Severity	Discovered	Confirmed	Fixed
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	1	1	0
Informational	0	0	0
Total	1	1	0

Detailed Findings

Low Severity

L-01 RequestsManagerV2::completeMint and RequestsManagerV2::completeBurn forwards de-allowlisted deposit tokens to treasury

Severity	Files	Status	Properties Violated
Low	RequestsManagerV2	Acknowledged	E-14

Description:

`removeAllowedToken` (`src/RequestsManagerV2.sol:149`) de-lists a deposit or withdrawal token immediately without checking whether any `CREATED` requests referencing that token are still open. Neither `completeMint` (`src/RequestsManagerV2.sol:274`) nor `completeBurn` (`src/RequestsManagerV2.sol:392`) carries an `allowedToken` guard at settlement time — the allowlist check is applied only at request creation via the `allowedToken` modifier on `requestMint` (line 210) and `requestBurn` (line 314).

This creates two reachable violation paths:

Path 1 — `completeMint` : treasury receives a de-allowlisted deposit token.

1. A provider calls `requestMint(tokenA, amount)` while `tokenA` is in the allowlist. The deposit is escrowed in the contract.
2. `DEFAULT_ADMIN_ROLE` calls `removeAllowedToken(tokenA)`. The mapping entry is set to `false` with no guard on pending requests.
3. `SERVICE_ROLE` calls `completeMint(id)`. The call succeeds — `IERC20(token).safeTransfer()` (`src/RequestsManagerV2.sol:298`) transfers the de-allowlisted deposit token to the treasury, increasing its balance.

Path 2 — `completeBurn` : treasury is drained of a de-allowlisted withdrawal token.

1. A provider calls `requestBurn(amount, tokenA)` while `tokenA` is in the allowlist. Issue tokens are escrowed.
2. `DEFAULT_ADMIN_ROLE` calls `removeAllowedToken(tokenA)`.
3. `SERVICE_ROLE` calls `completeBurn(id)`. The call succeeds — `IERC20(token).safeTransferFrom()` (`src/RequestsManagerV2.sol:425`) pulls the de-allowlisted withdrawal token out of the treasury, decreasing its balance.

In both cases the treasury balance of a token that is no longer in the allowlist changes, violating property E-14.

Recommended Mitigation:

Add an `allowedToken(request.token)` check at the top of both `completeMint` and `completeBurn` so that settlement reverts if the token has been de-allowlisted since the request was created:

```
function completeMint(uint256 _id) external onlyRole(SERVICE_ROLE) whenNotPaused mintRequestExist(_id) {
    MintRequest storage request = mintRequests[_id];
    _assertState(State.CREATED, request.state);
+   if (!allowedTokens[request.token]) revert TokenNotAllowed(request.token);
    ...
}

function completeBurn(uint256 _id) external onlyRole(SERVICE_ROLE) whenNotPaused burnRequestExist(_id) {
    BurnRequest storage request = burnRequests[_id];
    _assertState(State.CREATED, request.state);
+   if (!allowedTokens[request.token]) revert TokenNotAllowed(request.token);
    ...
}
```

Alternatively, `removeAllowedToken` can be made to revert if any `CREATED` mint or burn requests reference the token being de-listed. However, this requires an on-chain index of open requests per token and is significantly more expensive to implement.

Violated Property:

```
///  
// RMV2_E_14 - treasury balance of a non-allowlisted token never changes.  
rule treasuryNonAllowedTokenBalancePersists(env e, method f, calldataarg args)  
  filtered { f -> commonFilters(f) }  
{  
  setup(e);  
  require !allowedTokens(tokenA), "tokenA not in allowlist";  
  address treasury = treasuryAddress();  
  mathint balBefore = to_mathint(tokenA.balanceOf(treasury));  
  f(e, args);  
  assert to_mathint(tokenA.balanceOf(treasury)) == balBefore,  
    "treasury balance of a non-allowlisted token must not change";  
}
```

Violated Run Link: [RMV2_Escrow.conf](#)

Status:

Acknowledged.

Formal Verification

Formal Verification Methodology

Formal verification was used to reason about the `RequestsManagerV2`, `SimpleToken`, and `PriceStorage` contracts against explicitly written correctness properties. Rather than exercising a selected set of examples, as in unit tests or fuzz tests, the Certora Prover analyzes the compiled contracts symbolically and checks whether the specified properties hold for all relevant inputs, callers, contract states, and execution paths within the verification model.

The specifications were written in CVL, the Certora Verification Language. Each CVL rule captures an expected protocol guarantee, such as preserving escrow accounting invariants, enforcing role-based access control, respecting the three-state request lifecycle, or rejecting invalid settlement inputs.

The Solidity contracts, harnesses, and CVL specifications are then submitted to the prover, which translates them into logical constraints and attempts to either prove the property or produce a counterexample showing how it can be violated. This methodology complements testing by covering broad state spaces that are difficult to enumerate manually. Tests are useful for concrete scenarios and integration behavior, while formal verification is used here to check general safety properties across the protocol's critical flows.

Types of Properties

Valid State: invariants and global correctness conditions that should hold across all reachable states.

Variable Transition: rules proving that selected state variables change only in expected ways.

State Transition: authorization, gating, and lifecycle transition properties.

High-Level: end-to-end economic and business-logic properties.

Unit / Liveness: reachability and model sanity checks.

Negative / Anti-Property: offensive witnesses or expected-failure checks documenting what must not be assumed.

Verification Notations

Notation	Description
✓	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
×	A counter-example exists that violates one of the assertions of the rule.

General Assumptions and Simplifications

- We used Solidity Compiler version 0.8.28 to verify the protocol, without using the solc optimizer and without via-ir.
- `PRICE_STORAGE.lastPrice()` is summarized by a CVL ghost returning a symbolic `(price, timestamp)` pair. Price is treated as a free symbolic value; oracle staleness and price-update ordering are not checked (F-01 carve-out).
- `EIP-2612 permit` calls are summarized as `NONDET` to allow `requestMintWithPermit` / `requestBurnWithPermit` to reach a success path without modeling `ecrecover` signature verification.

- `SafeERC20.safeTransfer` and `safeTransferFrom` are summarized by CVL dispatch functions that route to the concrete linked token contracts (`SimpleTokenHarness` , `DummyERC20Permit` , `DummyERC20A`), keeping all token storage on real OZ storage and preventing unintended HAVOC
- Linked token contracts are modeled as standard 18-decimal, non-fee-on-transfer ERC-20s. Fee-on-transfer, rebase, and other non-standard behaviors are not verified.
- `emergencyWithdraw` solvency is not asserted. Only state-inertness is verified.
- Loop iterations are bounded at 2 (`optimistic_loop` , `loop_iter = 2`). Hashing is bounded at 128 bytes (`optimistic_hashing` , `hashing_length_bound = 128`).
- `SimpleToken.initialize(string,string)` is excluded from parametric rules in all SimpleToken specs; initialization correctness is verified separately via `ST-CORE-27` / `ST-CORE-28` .

Verification Properties

RequestsManagerV2 — Access Control

These properties verify that every privileged operation in `RequestsManagerV2` is gated by the correct role or caller identity. Coverage includes admin-only config setters, service-role completion, provider/admin cancellation rights, pause mechanics, allowlist invariants, and absence of privilege escalation.

Run Link

- **Audit Run Link:** [RMV2_AccessControl.conf](#)
- **Mitigation Run Link:** [RMV2_AccessControl.conf](#)

Properties

Code	Name	Description	Audit	Mitig	Issues
AC-1	<code>treasuryOnlyViaSetTreasuryByAdmin</code>	<code>treasuryAddress</code> changes only via <code>setTreasury</code> ; only <code>DEFAULT_ADMIN_ROLE</code> may call it.	✓	✓	
AC-2	<code>allowedTokensOnlyViaAddRemoveByAdmin</code>	<code>allowedTokens[t]</code> changes only via <code>add/removeAllowedToken</code> ; only <code>DEFAULT_ADMIN_ROLE</code> .	✓	✓	
AC-3	<code>mintFeeOnlyViaSetMintFeeByAdmin</code>	<code>mintFee</code> changes only via <code>setMintFee</code> ; only <code>DEFAULT_ADMIN_ROLE</code> .	✓	✓	
AC-4	<code>burnFeeOnlyViaSetBurnFeeByAdmin</code>	<code>burnFee</code> changes only via <code>setBurnFee</code> ; only <code>DEFAULT_ADMIN_ROLE</code> .	✓	✓	
AC-5	<code>mintTTLOnlyViaSetterByAdmin</code>	<code>mintRequestTTL</code> changes only via <code>setMintRequestTTL</code> ; only <code>DEFAULT_ADMIN_ROLE</code> .	✓	✓	
AC-6	<code>burnTTLOnlyViaSetterByAdmin</code>	<code>burnRequestTTL</code> changes only via <code>setBurnRequestTTL</code> ; only <code>DEFAULT_ADMIN_ROLE</code> .	✓	✓	
AC-7	<code>burnCancelWindowOnlyViaSetterByAdmin</code>	<code>burnCancelWindow</code> changes only via <code>setBurnCancelWindow</code> ; only <code>DEFAULT_ADMIN_ROLE</code> .	✓	✓	
AC-8	<code>pauseTransitionsAuthorized</code>	<code>paused()</code> toggles only via <code>pause/unpause</code> ; <code>PAUSER_ROLE</code> pauses, <code>DEFAULT_ADMIN_ROLE</code> unpauses.	✓	✓	
AC-9	<code>mintCompletionRequiresService</code>	<code>CREATED</code> → <code>COMPLETED</code> for a mint request requires <code>SERVICE_ROLE</code> .	✓	✓	
AC-10	<code>burnCompletionRequiresService</code>	<code>CREATED</code> → <code>COMPLETED</code> for a burn request requires <code>SERVICE_ROLE</code> .	✓	✓	
AC-11	<code>mintCancellationRequiresProviderOrAdmin</code>	<code>CREATED</code> → <code>CANCELLED</code> for a mint request requires caller == provider or <code>DEFAULT_ADMIN_ROLE</code> .	✓	✓	
AC-12	<code>burnCancellationRequiresProviderOrAdmin</code>	<code>CREATED</code> → <code>CANCELLED</code> for a burn request requires caller == provider or <code>DEFAULT_ADMIN_ROLE</code> .	✓	✓	
AC-13	<code>emergencyWithdrawRequiresAdmin</code>	only <code>DEFAULT_ADMIN_ROLE</code> can call <code>emergencyWithdraw</code> .	✓	✓	

Code	Name	Description	Audit	Mitig	Issues
AC-14	getRoleAdminImmutable	getRoleAdmin(role) is immutable; _setRoleAdmin never succeeds in OZ DefaultAdminRules.	✓	✓	
AC-15	serviceRoleChangeAuthorized	SERVICE_ROLE changes only via admin-gated grant/revoke or self-renounce.	✓	✓	
AC-16	pauserRoleChangeAuthorized	PAUSER_ROLE changes only via admin-gated grant/revoke or self-renounce.	✓	✓	
AC-17	noSelfEscalationToAdmin	a non-admin caller cannot gain DEFAULT_ADMIN_ROLE in one call.	✓	✓	
AC-18	mintFeeWithinMax	mintFee $\leq$ MAX_FEE at all times.	✓	✓	
AC-19	burnFeeWithinMax	burnFee $\leq$ MAX_FEE at all times.	✓	✓	
AC-20	burnRequestFeeWithinMax	every existing burn request's locked fee is $\leq$ MAX_FEE.	✓	✓	
AC-21	treasuryNeverZero	treasuryAddress is never the zero address.	✓	✓	
AC-22	issueTokenNeverAllowed	the issue token is never in the allowlist.	✓	✓	
AC-23	zeroAddressNeverAllowed	address(0) is never in the allowlist.	✓	✓	
AC-24	requestMintRevertsWhenPaused	requestMint reverts when paused.	✓	✓	
AC-25	requestBurnRevertsWhenPaused	requestBurn reverts when paused.	✓	✓	
AC-26	completeMintRevertsWhenPaused	completeMint reverts when paused.	✓	✓	
AC-27	completeBurnRevertsWhenPaused	completeBurn reverts when paused.	✓	✓	
AC-28	mintRequestTokenIsAllowed	token stored in any existing mint request is always an allowed token.	✓	✓	
AC-29	burnRequestTokenIsAllowed	token stored in any existing burn request is always an allowed token.	✓	✓	
AC-30	completeMintRejectsExpired	completeMint reverts when the request is past its TTL.	✓	✓	
AC-31	completeBurnRejectsExpired	completeBurn reverts when the request is past its TTL.	✓	✓	
AC-32	cancelBurnOnlyWithinWindow	cancelBurn (provider path) succeeds only within burnCancelWindow.	✓	✓	
AC-33	requestMintWithPermitSuccessSubset	requestMintWithPermit succeeds only when base requestMint preconditions hold.	✓	✓	
AC-34	requestBurnWithPermitSuccessSubset	requestBurnWithPermit succeeds only when base requestBurn preconditions hold.	✓	✓	
AC-35	allowedTokenHas18Decimals	every token in the allowlist has exactly 18 decimals.	✓	✓	
AC-36	cancelMintNotGatedByAllowlist	cancelMint is not gated by the allowlist; a de-allowlisted token does not block cancellation.	✓	✓	
AC-37	cancelBurnNotGatedByAllowlist	cancelBurn is not gated by the allowlist; a de-allowlisted token does not block cancellation.	✓	✓	

RequestsManagerV2 — Idempotency Keys

These properties verify that the on-chain idempotency keys used by `completeMint` and `completeBurn` are collision-free. Coverage includes injectivity of mint and burn key derivation, disjointness between mint and burn namespaces, and separation from V1 legacy keys by both tag byte and preimage length.

Run Link

- **Audit Run Link:** [RMV2_Idempotency.conf](#)
- **Mitigation Run Link:** [RMV2_Idempotency.conf](#)

Properties

Code	Name	Description	Audit	Mitig	Issues
IDP-1	mintKeyInjective	keyMint is injective over ids: $i \neq j \Rightarrow \text{keyMint}(i) \neq \text{keyMint}(j)$.	✓	✓	
IDP-2	burnKeyInjective	keyBurn is injective over ids: $i \neq j \Rightarrow \text{keyBurn}(i) \neq \text{keyBurn}(j)$.	✓	✓	
IDP-3	mintBurnKeyDisjoint	Mint and burn V2 keys are disjoint: no $\text{keyMint}(i)$ equals any $\text{keyBurn}(j)$.	✓	✓	
IDP-4	v2DisjointFromLegacy_byTagCase	V2 lowercase tags ('m'/'b') differ from legacy uppercase ('M'/'B') at every id.	✓	✓	
IDP-5	v2DisjointFromLegacy_byLength	V2 preimage is exactly 36 bytes; legacy is strictly longer due to non-empty product prefix.	✓	✓	

RequestsManagerV2 — Escrow

These properties verify the token-flow integrity of `RequestsManagerV2`. Coverage includes settlement conservation for mint and burn, exact inflow on deposit, native ETH cannot accumulate, treasury protection for non-allowlisted tokens, zero-address token balance safety, full refund on all cancellation paths, and the emergencyWithdraw rug-path consequences: once the deposit token is drained, `completeMint`, `cancelMint`, and `adminCancelMint` all revert; once the issue token is drained, `completeBurn`, `cancelBurn`, and `adminCancelBurn` all revert — making escrowed funds irrecoverable through any path on both sides.

Run Link

- **Audit Run Link:** [RMV2_Escrow.conf](#)
- **Mitigation Run Link:** [RMV2_Escrow.conf](#)

Properties

Code	Name	Description	Audit	Mitig	Issues
E-1	mintEscrowBacked	Per-deposit-token escrow backing lower bound.	✓	✓	
E-2	burnEscrowBacked	Issue-token escrow backing lower bound.	✓	✓	
E-3	completeMint_settlementConservation	<code>completeMint</code> sends exactly <code>request.amount</code> deposit token to treasury and mints exactly <code>mintAmount</code> to provider.	✓	✓	
E-4	completeBurn_settlementConservation	<code>completeBurn</code> burns exactly <code>request.amount</code> issue token and sends exactly <code>withdrawalAmount</code> from treasury to provider.	✓	✓	
E-5	requestMintInflowIntegrity	<code>requestMint</code> pulls in exactly <code>request.amount</code> of the deposit token (non-FoT token).	✓	✓	
E-6	requestBurnInflowIntegrity	<code>requestBurn</code> pulls in exactly <code>request.amount</code> of the issue token.	✓	✓	
E-7	noNativeEthStuck	Manager native ETH balance stays 0 across any call.	✓	✓	
E-8	requestMintRevertsWhenInsufficientBalance	<code>requestMint</code> reverts when amount exceeds the provider's deposit-token balance.	✓	✓	
E-9	onlyMsgSenderBalanceCanDecrease	No RMV2 call may decrease a third party's tokenA balance; only <code>msg.sender</code> 's balance can decrease.	✓	✓	
E-10	requestBurnRevertsWhenInsufficientBalance	<code>requestBurn</code> reverts when amount exceeds the provider's issue-token balance.	✓	✓	
E-11	onlyMsgSenderIssueTokenBalanceCanDecrease	No RMV2 call may decrease a third party's issue-token balance; only <code>msg.sender</code> 's balance can decrease.	✓	✓	
E-12	zeroAddressHoldsNoIssueToken	<code>address(0)</code> never holds any issue token.	✓	✓	
E-13	zeroAddressHoldsNoTokenA	<code>address(0)</code> never holds any tokenA.	✓	✓	

Code	Name	Description	Audit	Mitig	Issues
E-14	treasuryNonAllowedTokenBalancePersists	Treasury balance of a non-allowlisted token never changes.	×	ACK	L-01, ACK
E-15	treasuryTokenAUnchangedIfPriceZero	Treasury's tokenA balance cannot change when the oracle price is zero.	✓	✓	
E-16	cancelMintFullRefund	cancelMint refunds exactly request.amount to the provider; no fee haircut.	✓	✓	
E-17	adminCancelMintFullRefund	adminCancelMint refunds exactly request.amount to the provider; no fee haircut.	✓	✓	
E-18	cancelBurnFullRefund	cancelBurn refunds exactly request.amount of issue token to the provider; no fee haircut.	✓	✓	
E-19	adminCancelBurnFullRefund	adminCancelBurn refunds exactly request.amount of issue token to the provider; no fee haircut.	✓	✓	
E-20	completeMintRevertsAfterEmergencyWithdraw	completeMint reverts after emergency-Withdraw drains the deposit token; the rug path makes settlement impossible.	✓	✓	
E-21	cancelMintRevertsAfterEmergencyWithdraw	cancelMint reverts after emergency-Withdraw drains the deposit token; providers cannot recover escrowed funds.	✓	✓	
E-22	adminCancelMintRevertsAfterEmergencyWithdraw	adminCancelMint reverts after emergencyWithdraw drains the deposit token; even the admin cancel path cannot return escrowed funds.	✓	✓	
E-23	emergencyWithdrawDrainsFullBalance	emergencyWithdraw leaves exactly zero deposit token balance in the contract; the entire balance is transferred to the admin caller.	✓	✓	
E-24	completeBurnRevertsAfterEmergencyWithdrawIssueToken	completeBurn reverts after emergency-Withdraw drains the issue token; the burn-escrow rug path makes settlement impossible.	✓	✓	
E-25	cancelBurnRevertsAfterEmergencyWithdrawIssueToken	cancelBurn reverts after emergencyWithdraw drains the issue token; providers cannot recover escrowed issue tokens.	✓	✓	
E-26	adminCancelBurnRevertsAfterEmergencyWithdrawIssueToken	adminCancelBurn reverts after emergencyWithdraw drains the issue token; even the admin cancel path cannot return escrowed issue tokens.	✓	✓	

RequestsManagerV2 — State Machine

These properties verify the request lifecycle state machine. Coverage includes terminal-state absorption (COMPLETED/CANCELLED are absorbing), transitions only out of CREATED, wrong-state rejection, CEI ordering (state written before any token transfer), pause-independence of admin cancel paths, prevention of double-settlement, and TTL-independence of provider cancel paths (both `cancelMint` and `cancelBurn` remain available after TTL expiry).

Run Link

- **Audit Run Link:** [RMV2_StateMachine.conf](#)
- **Mitigation Run Link:** [RMV2_StateMachine.conf](#)

Properties

Code	Name	Description	Audit	Mitig	Issues
SM-1	mintTerminalAbsorption	COMPLETED/CANCELLED mint request stays in that terminal state forever.	✓	✓	

Code	Name	Description	Audit	Mitig	Issues
SM-2	burnTerminalAbsorption	COMPLETED/CANCELLED burn request stays in that terminal state forever.	✓	✓	
SM-3	mintTransitionsOnlyFromCreated	A mint request can only transition out of CREATED.	✓	✓	
SM-4	burnTransitionsOnlyFromCreated	A burn request can only transition out of CREATED.	✓	✓	
SM-5	mintWrongStateReverts	completeMint/cancelMint/adminCancelMint revert when not CREATED.	✓	✓	
SM-6	burnWrongStateReverts	completeBurn/cancelBurn/adminCancelBurn revert when not CREATED.	✓	✓	
SM-7	mintCEIStateBeforeTransfer_complete	CEI: completeMint writes state before any transfer.	✓	✓	
SM-8	mintCEIStateBeforeTransfer_cancel	CEI: cancelMint writes state before any transfer.	✓	✓	
SM-9	mintCEIStateBeforeTransfer_adminCancel	CEI: adminCancelMint writes state before any transfer.	✓	✓	
SM-10	burnCEIStateBeforeTransfer_complete	CEI: completeBurn writes state before any transfer.	✓	✓	
SM-11	burnCEIStateBeforeTransfer_cancel	CEI: cancelBurn writes state before any transfer.	✓	✓	
SM-12	burnCEIStateBeforeTransfer_adminCancel	CEI: adminCancelBurn writes state before any transfer.	✓	✓	
SM-13	adminCancelMintPauseIndependent	adminCancelMint does not revert due to pause state.	✓	✓	
SM-14	adminCancelBurnPauseIndependent	adminCancelBurn does not revert due to pause state.	✓	✓	
SM-15	G_emergencyWithdrawStateInert	emergencyWithdraw does not touch any request state or field.	✓	✓	
SM-16	adminCancelBurnIgnoresCancelWindow	adminCancelBurn ignores burnCancelWindow; admin can cancel even after the window has elapsed.	✓	✓	
SM-17	cancelMintTTLIndependent	cancelMint is not gated by mintRequestTTL; provider can cancel a CREATED mint even after TTL has elapsed.	✓	✓	
SM-18	cancelBurnTTLIndependent	cancelBurn is not gated by burnRequestTTL; provider can cancel a CREATED burn within the cancel window even after TTL has elapsed.	✓	✓	
SM-19	noDoubleCompleteMint	A second completeMint for the same id always reverts (COMPLETED is terminal).	✓	✓	
SM-20	noDoubleCompleteBurn	A second completeBurn for the same id always reverts (COMPLETED is terminal).	✓	✓	

RequestsManagerV2 — Structural

These properties verify the structural integrity of request storage. Coverage includes monotonic and bounded counters, fresh-slot guarantee (no id reuse), write-targeting of `requestMint` / `requestBurn`, field immutability after creation, non-existent-request revert, V1/V2 id-space separation via `INITIAL_COUNTER`, and fee-isolation on `setBurnFee`.

Run Link

- **Audit Run Link:** [RMV2_Structural.conf](#)
- **Mitigation Run Link:** [RMV2_Structural.conf](#)

Properties

Code	Name	Description	Audit	Mitig	Issues
STR-1	mintCounterMonotonic	mintRequestsCounter never decreases and increases by at most 1 per call.	✓	✓	

Code	Name	Description	Audit	Mitig	Issues
STR-2	burnCounterMonotonic	burnRequestsCounter never decreases and increases by at most 1 per call.	✓	✓	
STR-3	mintFreshSlot	Any $id \geq \text{mintRequestsCounter}$ has an empty slot (no mint id reuse).	✓	✓	
STR-4	burnFreshSlot	Any $id \geq \text{burnRequestsCounter}$ has an empty slot (no burn id reuse).	✓	✓	
STR-5	requestMintWriteTargeting	requestMint writes only the pre-call counter slot, sets it CREATED/msg.sender, increments counter by one, touches no other id.	✓	✓	
STR-6	requestBurnWriteTargeting	requestBurn writes only the pre-call counter slot, sets it CREATED/msg.sender, increments counter by one, touches no other id.	✓	✓	
STR-7	mintFieldImmutability	provider/createdAt/token/amount of a mint request are fixed after creation; only state may change.	✓	✓	
STR-8	burnFieldImmutability	provider/createdAt/token/amount/price/fee of a burn request are fixed after creation; only state may change.	✓	✓	
STR-9	mintNonExistentRequestReverts	completeMint/cancelMint/adminCancelMint revert when the mint request does not exist (provider == 0).	✓	✓	
STR-10	burnNonExistentRequestReverts	completeBurn/cancelBurn/adminCancelBurn revert when the burn request does not exist (provider == 0).	✓	✓	
STR-11	mintCounterAtLeastInitial	mintRequestsCounter never falls below INITIAL_COUNTER, preserving V1 id-separation.	✓	✓	
STR-12	burnCounterAtLeastInitial	burnRequestsCounter never falls below INITIAL_COUNTER, preserving V1 id-separation.	✓	✓	
STR-13	mintRequestAmountNonZero	Every existing mint request (provider $\neq$ 0) has a non-zero amount.	✓	✓	
STR-14	burnRequestAmountNonZero	Every existing burn request (provider $\neq$ 0) has a non-zero amount.	✓	✓	
STR-15	immutablesNeverChange	ISSUE_TOKEN_ADDRESS never changes after construction.	✓	✓	
STR-16	priceStorageImmutable	PRICE_STORAGE never changes after construction.	✓	✓	
STR-17	setBurnFeeDoesNotTouchPendingBurns	setBurnFee does not mutate locked fees of already-created burn requests.	✓	✓	

RequestsManagerV2 — Settlement Mathematics

These properties verify the settlement mathematics for mint and burn operations. Coverage includes anti-inflation ceilings, protocol-favoring truncation, fee and price monotonicity, intermediate overflow safety, supply change attribution, dust-revert behavior, and no-free-value guarantees.

Run Link

- **Audit Run Link:** [RMV2_Settlement.conf](#)
- **Mitigation Run Link:** [RMV2_Settlement.conf](#)

Properties

Code	Name	Description	Audit	Mitig	Issues
S-1	mintPreFeeBound	Pre-fee anti-inflation ceiling: $\text{post-fee mint amount} \leq \text{deposit} * \text{PRECISION} / \text{price}$.	✓	✓	
S-2	mintRoundsDownTowardProtocol	Mint rounding always favors the protocol (user gets floor).	✓	✓	
S-3	burnRoundsDownTowardProtocol	Burn rounding always favors the protocol (user gets floor).	✓	✓	

Code	Name	Description	Audit	Mitig	Issues
S-4	mintFeeMonotonic	Mint output is monotone non-increasing in fee.	✓	✓	
S-5	mintPriceMonotonic	Mint output is monotone non-increasing in price.	✓	✓	
S-6	burnFeeMonotonic	Burn output is monotone non-increasing in fee.	✓	✓	
S-7	burnPriceMonotonic	Burn output is monotone non-decreasing in price.	✓	✓	
S-8	mintIntermediatesFitUint256	Mint intermediates fit uint256; (PRECISION - fee) never underflows.	✓	✓	
S-9	burnIntermediatesFitUint256	Burn intermediates fit uint256; (PRECISION - fee) never underflows.	✓	✓	
S-10	completeMint_antiInflation	completeMint raises totalSupply by exactly the computed amount, bounded by the anti-inflation ceiling.	✓	✓	
S-11	onlyCompleteMintIncreasesSupply	Only completeMint may increase the issue-token totalSupply.	✓	✓	
S-12	onlyCompleteBurnDecreasesSupply	Only completeBurn may decrease the issue-token totalSupply.	✓	✓	
S-13	completeBurn_settlement	completeBurn value conservation: exact settlement at min(locked,current) price.	✓	✓	
S-14	completeMint_zeroOut_revertsPreservesState	Mint dust rounds to zero => revert ZeroAmountOut, state stays CREATED.	✓	✓	
S-15	completeBurn_zeroOut_revertsPreservesState	Burn dust rounds to zero => revert ZeroAmountOut, state stays CREATED.	✓	✓	
S-16	noFreeMint	No free value: non-zero mint output requires non-zero deposit.	✓	✓	
S-17	noFreeBurn	No free value: non-zero burn output requires non-zero amount.	✓	✓	

SimpleToken — Core (Idempotency & Access Control)

These properties verify the idempotency, access control, and initialization mechanics of `SimpleToken`. Coverage includes write-once idempotency keys, key namespace isolation between mint and burn, supply change role gating, `DefaultAdminRules` coupling, permit deadline and nonce correctness, and single-initialization enforcement.

Run Link

- **Audit Run Link:** [SimpleToken_Core.conf](#)
- **Mitigation Run Link:** [SimpleToken_Core.conf](#)

Properties

Code	Name	Description	Audit	Mitig	Issues
ST-CORE-1	mintIdsMonotonic	mintIds[key] once set to true is never reset.	✓	✓	
ST-CORE-2	burnIdsMonotonic	burnIds[key] once set to true is never reset.	✓	✓	
ST-CORE-3	keyedMintUsedKeyReverts	Keyed mint with an already-used key reverts.	✓	✓	
ST-CORE-4	keyedBurnUsedKeyReverts	Keyed burn with an already-used key reverts.	✓	✓	
ST-CORE-5	keyedMintOneShot	Keyed mint is one-shot: a second call with the same key always reverts.	✓	✓	
ST-CORE-6	keyedBurnOneShot	Keyed burn is one-shot: a second call with the same key always reverts.	✓	✓	
ST-CORE-7	keyedMintWriteIsolation	Keyed mint writes exactly its own mintIds key; no collateral writes.	✓	✓	
ST-CORE-8	keyedBurnWriteIsolation	Keyed burn writes exactly its own burnIds key; no collateral writes.	✓	✓	
ST-CORE-9	keyedMintDoesNotTouchBurnIds	Keyed mint never writes burnIds.	✓	✓	
ST-CORE-10	keyedBurnDoesNotTouchMintIds	Keyed burn never writes mintIds.	✓	✓	

Code	Name	Description	Audit	Mitig	Issues
ST-CORE-11	sameKeyReusableAcrossNamespaces	The same key value is independently consumable across the mint and burn namespaces.	✓	✓	
ST-CORE-12	mintIdsWrittenOnlyByKeyedMint	mintIds[key] may change only via keyed mint(bytes32,address,uint256).	✓	✓	
ST-CORE-13	burnIdsWrittenOnlyByKeyedBurn	burnIds[key] may change only via keyed burn(bytes32,address,uint256).	✓	✓	
ST-CORE-14	supplyIncreaseRequiresServiceRole	totalSupply may increase only when caller holds SERVICE_ROLE.	✓	✓	
ST-CORE-15	supplyDecreaseRequiresServiceRole	totalSupply may decrease only when caller holds SERVICE_ROLE.	✓	✓	
ST-CORE-16	serviceRoleAdminNeverChanges	getRoleAdmin(SERVICE_ROLE) is immutable; _setRoleAdmin never succeeds.	✓	✓	
ST-CORE-17	serviceRoleAdminIsDefaultAdmin	getRoleAdmin(SERVICE_ROLE) == DEFAULT_ADMIN_ROLE invariant.	✓	✓	
ST-CORE-18	defaultAdminRoleCoupledToSlot	hasRole(DEFAULT_ADMIN_ROLE, a) iff (a ≠ 0 and a == defaultAdmin()) OZ coupling.	✓	✓	
ST-CORE-19	atMostOneDefaultAdmin	At most one DEFAULT_ADMIN_ROLE holder at any time.	✓	✓	
ST-CORE-20	grantDefaultAdminReverts	grantRole(DEFAULT_ADMIN_ROLE, ...) always reverts (DefaultAdminRules).	✓	✓	
ST-CORE-21	revokeDefaultAdminReverts	revokeRole(DEFAULT_ADMIN_ROLE, ...) always reverts (DefaultAdminRules).	✓	✓	
ST-CORE-22	serviceRoleNoEscalation	SERVICE_ROLE holder cannot self-escalate to DEFAULT_ADMIN_ROLE in one call.	✓	✓	
ST-CORE-23	acceptOnlyGrantsToScheduledPendingAdmin	acceptDefaultAdminTransfer can only grant DEFAULT_ADMIN_ROLE to the scheduled pendingDefaultAdmin.	✓	✓	
ST-CORE-24	permitRespectsDeadline	permit reverts once block.timestamp > deadline.	✓	✓	
ST-CORE-25	permitSuccessPostCondition	Successful permit sets allowance == value and increments nonces(owner) by exactly 1.	✓	✓	
ST-CORE-26	nonceMonotonicAndOnlyPermit	nonces(owner) is monotonic non-decreasing and changes only via permit.	✓	✓	
ST-CORE-27	initializeAtMostOnce	initialize is callable at most once (sequential form).	✓	✓	
ST-CORE-28	initializeRevertsWhenInitialized	initialize reverts when the contract is already initialized.	✓	✓	

SimpleToken — ERC-20

These properties verify the standard ERC-20 behavior of `SimpleToken`. Coverage includes the sum-of-balances invariant, supply change attribution, per-account balance bound, transfer and `transferFrom` conservation, self-transfer identity, allowance decrement, zero-address rejection, and correct mint/burn effects for both keyed and keyless overloads.

Run Link

- **Audit Run Link:** [SimpleToken_ERC20.conf](#)
- **Mitigation Run Link:** [SimpleToken_ERC20.conf](#)

Properties

Code	Name	Description	Audit	Mitig	Issues
ST-ERC20-1	totalSupplyIsSumOfBalances	sum(balanceOf) == totalSupply at every reachable state.	✓	✓	
ST-ERC20-2	totalSupplyChangesOnlyViaMintBurn	totalSupply changes only via the four mint/burn overloads.	✓	✓	

Code	Name	Description	Audit	Mitig	Issues
ST-ERC20-3	balanceLeTotalSupply	balanceOf(a) $\leq$ totalSupply for every account.	✓	✓	
ST-ERC20-4	transferConservesSupply	transfer conserves totalSupply and moves exactly amt from sender to receiver.	✓	✓	
ST-ERC20-5	transferFromConservesSupply	transferFrom conserves totalSupply and moves exactly amt from/to.	✓	✓	
ST-ERC20-6	selfTransferIdentity	Self-transfer leaves sender balance unchanged.	✓	✓	
ST-ERC20-7	transferFromDecrementsAllowance	transferFrom decrements allowance by amt unless infinite approval.	✓	✓	
ST-ERC20-8	transferRejectsZeroAddress	transfer reverts when to or from is the zero address.	✓	✓	
ST-ERC20-9	transferFromRejectsZeroAddress	transferFrom reverts when from or to is the zero address.	✓	✓	
ST-ERC20-10	keylessMintEffect	Keyless mint raises balanceOf(account) and totalSupply each by amt; reverts if account == 0.	✓	✓	
ST-ERC20-11	keyedMintEffect	Keyed mint raises balanceOf(account) and totalSupply each by amt; reverts if account == 0.	✓	✓	
ST-ERC20-12	keylessBurnEffect	Keyless burn lowers balanceOf(account) and totalSupply each by amt; reverts if account == 0.	✓	✓	
ST-ERC20-13	keyedBurnEffect	Keyed burn lowers balanceOf(account) and totalSupply each by amt; reverts if account == 0.	✓	✓	
ST-ERC20-14	burnRejectsInsufficientBalance	burn reverts when account == 0 or amt > balance.	✓	✓	

PriceStorage

These properties verify the price oracle mechanics of `PriceStorage`. Coverage includes write-once per-key immutability, upward and downward band enforcement, timestamp monotonicity, `lastPrice` mirroring, service-role gating on writes, admin-only bound updates, bound range invariants, initialization correctness, and zero-key rejection.

Run Link

- **Audit Run Link:** [PriceStorage.conf](#)
- **Mitigation Run Link:** [PriceStorage.conf](#)

Properties

Code	Name	Description	Audit	Mitig	Issues
P-1	priceKeyWriteOnce	Once prices[key] is written (timestamp $\neq$ 0), neither field ever changes.	✓	✓	
P-2	setPriceUpwardBound	lastPrice.price never exceeds prev * (1 + upperBoundPercentage / 1e18).	✓	✓	
P-3	setPriceDownwardBound	lastPrice.price never falls below prev * (1 - lowerBoundPercentage / 1e18).	✓	✓	
P-4	lastPriceTimestampMonotonicOnWrite	lastPrice.timestamp equals block.timestamp on every write and never decreases.	✓	✓	
P-5	lastPriceTimestampOnlySetPrice	Only setPrice may mutate lastPrice.timestamp.	✓	✓	
P-6	lastPriceMirrorsWrittenKey	After setPrice, lastPrice mirrors the key written in the same call.	✓	✓	
P-7	priceNonZeroWhenSet	prices[key].price is never zero once the slot is written.	✓	✓	
P-8	lastPriceZeroIffUnwritten	lastPrice fields are either both zero or both non-zero.	✓	✓	

Code	Name	Description	Audit	Mitig	Issues
P-9	onlyServiceRoleWritesPrice	Any mutation to prices or lastPrice requires SERVICE_ROLE.	✓	✓	
P-10	onlyAdminChangesBounds	Changing upperBoundPercentage or lowerBoundPercentage requires DEFAULT_ADMIN_ROLE.	✓	✓	
P-11	upperBoundInRange	After initialization upperBoundPercentage is in (0, 1e18].	✓	✓	
P-12	lowerBoundInRange	After initialization lowerBoundPercentage is in (0, 1e18].	✓	✓	
P-13	initializeAtMostOnce	initialize can only be called once; a second call always reverts.	✓	✓	
P-14	initializeSetsVersion	initialize sets the version to exactly 1.	✓	✓	
P-15	serviceRoleAdminNeverChanges	getRoleAdmin(SERVICE_ROLE) is permanently DEFAULT_ADMIN_ROLE.	✓	✓	
P-16	noServiceToAdminEscalation	A SERVICE_ROLE-only caller cannot grant DEFAULT_ADMIN_ROLE to any account.	✓	✓	
P-17	lastPriceValueOnlySetPrice	lastPrice.price can only be changed by setPrice.	✓	✓	
P-18	firstPriceAlwaysAccepted	When no price has been written yet, setPrice accepts any non-zero price (band check is skipped).	✓	✓	
P-19	zeroKeyNeverSet	prices[bytes32(0)] is never written; the zero key is permanently rejected.	✓	✓	

Setup and Executions

The Certora Prover can be run remotely using Certora's cloud infrastructure or locally from a source build. For this engagement, the report metadata records Certora Prover version **8.16.1**. The commands below assume you are inside the repository root, where `certora/` is a direct child. No source code modification is required — all external calls and complex internals are handled through CVL summaries and ghost-based modeling within the spec files.

Common Setup

The first four steps are shared by remote and local execution.

Recommended: Use a Python virtual environment to isolate Certora dependencies from your system Python.

```
python3 -m venv .venv
source .venv/bin/activate
```

All subsequent `pip` / `pipx` installs should be run inside this environment.

Step 1 - Install Java (JDK 21)

```
sudo apt update
sudo apt install -y openjdk-21-jdk
java -version
```

Step 2 - Install pipx

`pipx` installs Python CLI tools in isolated environments, which keeps Certora dependencies separate from the repo's Python packages. See the [pipx documentation](#) for platform-specific notes.

```
sudo apt install -y pipx
pipx ensurepath
exec "$SHELL"
```

Step 3 - Install the Certora CLI

```
pipx install certora-cli==8.16.1
certoraRun --version
```

Step 4 - Install the Solidity Compiler

```
pipx install solc-select
solc-select install 0.8.28
solc-select use 0.8.28
solc --version
```

Remote Execution

A Certora key is required for cloud runs. Certora provides keys through its [Discord](#) and website.

```
echo "export CERTORAKEY=<your_certora_api_key>" >> ~/.bashrc
source ~/.bashrc
```

Run a configuration against the cloud backend:


```
certoraRun certora/conf/RMV2_Escrow.conf --server production
```

Local Execution

For a local build matching this engagement, follow the [CertoraProver repository](#). Once installed, the local Prover is used before the cloud backend unless `--server production` is passed.

Install Local-Prover Prerequisites

```
# JDK 21
sudo apt install -y openjdk-21-jdk

# SMT solvers: z3 and cvc5 are required; place their binaries on PATH.
# z3: https://github.com/Z3Prover/z3/releases
# cvc5: https://github.com/cvc5/cvc5/releases

# LLVM tools
sudo apt install -y llvm

# Rust 1.81.0+
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
cargo install rustfilt

# Optional, for visual reports
sudo apt install -y graphviz
```

Build CertoraProver v8.16.1

```
export CERTORA="$HOME/CertoraProver/target/installed/"
mkdir -p "$CERTORA"
export PATH="$CERTORA:$PATH"

git clone --recurse-submodules https://github.com/Certora/CertoraProver.git
cd CertoraProver
git checkout tags/8.16.1
./gradlew assemble
```

Verify the Local Installation

```
certoraRun.py -h
cd Public/TestEVM/Counter
certoraRun counter.conf
```

Running Avant RequestManagerV2 Verification

The verification suite is organized based on the contract they verify. All configuration files can be found in `certora/conf`.

Project Structure

```
certora/
├── conf/                                # Certora configuration files
│   ├── PriceStorage.conf
│   ├── RMV2_AccessControl.conf
│   ├── RMV2_Escrow.conf
│   ├── RMV2_Idempotency.conf
│   ├── RMV2_Settlement.conf
│   ├── RMV2_StateMachine.conf
│   ├── RMV2_Structural.conf
│   └── SimpleToken_Core.conf
```

```

├── SimpleToken_ERC20.conf
├── harness/                                # Harness contracts exposing internal state
│   ├── DummyERC20A.sol
│   ├── DummyERC20.sol
│   ├── DummyERC20Permit.sol
│   ├── PriceStorageHarness.sol
│   ├── RequestsManagerV2Harness.sol
│   └── SimpleTokenHarness.sol
├── specs/                                # CVL specification files
│   ├── PriceStorage.spec
│   ├── RMV2_AccessControl.spec
│   ├── RMV2_Escrow.spec
│   ├── RMV2_Idempotency.spec
│   ├── RMV2_Settlement.spec
│   ├── RMV2_StateMachine.spec
│   ├── RMV2_Structural.spec
│   ├── SimpleToken_Core.spec
│   ├── SimpleToken_ERC20.spec
│   └── utils/
│       └── RMV2_Base.spec                # Shared ghosts, hooks, and helpers

```

Run the commands below from the repository root.

Run the Full Suite

```

chmod +x certora/scripts/certora_run_all.sh
bash certora/scripts/certora_run_all.sh

```

Run All Configurations

```
certoraRun certora/conf/<config>.conf
```

For example:

```

# RequestsManagerV2
certoraRun certora/conf/RMV2_AccessControl.conf
certoraRun certora/conf/RMV2_Idempotency.conf
certoraRun certora/conf/RMV2_Escrow.conf
certoraRun certora/conf/RMV2_StateMachine.conf
certoraRun certora/conf/RMV2_Structural.conf
certoraRun certora/conf/RMV2_Settlement.conf

# SimpleToken
certoraRun certora/conf/SimpleToken_Core.conf
certoraRun certora/conf/SimpleToken_ERC20.conf

# PriceStorage
certoraRun certora/conf/PriceStorage.conf

```

Run One Rule While Debugging

```
certoraRun certora/conf/<config>.conf --rule <ruleName>
```

For example:

```

certoraRun certora/conf/RMV2_Escrow.conf --rule mintEscrowBacked
certoraRun certora/conf/RMV2_StateMachine.conf --rule mintTerminalAbsorption
certoraRun certora/conf/RMV2_AccessControl.conf --rule onlyAdminCanPause

```

Compilation-Only Checks

Use compilation-only checks before long runs or after touching imports, summaries, harnesses, or config paths.

```
certoraRun certora/conf/<config>.conf --compilation_steps_only
```

For example:

```
certoraRun certora/conf/RMV2_Escrow.conf --compilation_steps_only  
certoraRun certora/conf/RMV2_AccessControl.conf --compilation_steps_only  
certoraRun certora/conf/PriceStorage.conf --compilation_steps_only
```

Resources

To learn more about Certora formal verification, the resources below are useful for onboarding, writing stronger CVL properties, and debugging prover behavior.

- [Updraft Assembly & Formal Verification Course](#) - Comprehensive video course covering assembly and formal verification from the ground up.
- [RareSkills Certora Book](#) - Structured tutorial covering CVL syntax, specification patterns, and common pitfalls.
- [Certora Tutorials](#) - Official Certora tutorials for guided Prover, CVL, config, invariant, and debugging workflows.
- [Certora Prover Documentation](#) - Official Prover reference covering installation, CLI options, CVL, approximations, reports, and dashboard usage.

Disclaimer

This report reflects the results of a formal verification effort performed on a specific version of the codebase provided by the client. Formal verification increases confidence in the correctness of smart-contract behavior with respect to the specified rules; however, **it cannot guarantee the complete absence of bugs, vulnerabilities, or unexpected behavior.**

This document does not constitute legal, financial, or investment advice. The authors and reviewers assume no liability for the use of the verified smart contracts in production or for any losses arising from their deployment or interaction.